

大 作 业

揭棋对弈程序设计

公共通信协议规范

Interface Protocol Specification • v3.1

小组成员

张恒基（组长）

2024211301

2024210926

陈艺博

2024211302

2024210959

秦博宇

2024211302

2024210940

陈雨飞

2024211304

2024211005

项目代号：Unveil

2026 年 5 月 30 日

目录

1. 基础约定	1
1.1. 术语定义	1
1.2. 技术约定	1
2. 坐标系	2
2.1. 坐标定义	2
2.2. 坐标转换公式	3
3. 棋子类型编码	3
3.1. 老师 JSON piece 枚举映射 (§ 3.2)	3
3.2. 内部编码 ↔ JSON ↔ 中文三向对照 (§ 3.3)	4
4. 颜色编码	4
5. Move 对象规范	5
5.1. 类定义	5
5.2. 字段规则	5
5.3. 翻子随机性机制	5
5.4. JSON move 对象格式 (§ 5.4)	6
5.5. JSON moveResult 对象 (§ 5.5)	6
6. WebSocket + JSON 通信协议	6
6.1. 传输层约定 (§ 6.1)	6
6.2. 依赖库版本 (§ 6.2)	7
6.3. 消息格式总览 (§ 6.3)	7
6.4. 客户端 → 服务器消息 (§ 6.4)	7
6.5. 服务器 → 客户端消息 (§ 6.5)	9
6.6. 公共数据结构 (§ 6.6)	12
6.6.1. initialBoard	12
6.6.2. move 对象 (JSON)	12
6.6.3. moveResult 对象	12
6.7. 错误码 (§ 6.7)	13
6.7.1. gameOver / timeout 原因 (reason)	13
6.8. 典型 JSON 示例	13
6.8.1. 匹配与先手协商	13
6.8.2. 走子与翻子	14
6.8.3. 超时处理	15
6.8.4. 正常结束	15
6.8.5. 心跳	15
6.9. 本组扩展消息	15
7. 典型通信流程	15

7.1. 正常对弈时序	16
7.2. 非法着法	16
7.3. 超时判负	17
7.4. 认输	17
7.5. 先手协商时序 (requestFirstHand)	17
7.6. 断线判负	18
8. 暗子走法与虚拟类型	18
8.1. 虚拟类型机制	18
8.2. 明子强化规则	19
9. 胜负与和棋判定	19
9.1. 胜负条件	19
9.2. 和棋条件	19
9.3. 长将 / 长捉判定	20
9.4. 超时判定公式	20
10. 棋谱记录格式	20
10.1. 每步棋的记法	20
10.2. 棋子类型的棋谱名称	21
10.3. 服务器棋谱存储	21
11. 多盘对弈	21
11.1. 实现要点	21
11.2. 跨组兼容	21
12. 组间联调检查清单	22
13. 待老师确认的开放问题	22
13.1. 规则与胜负判定	23
13.2. 翻子、随机与棋谱	24
13.3. 网络、计时与互操作	24
13.4. AI 博弈与评估 (选做 AI 部分的组可一并裁定)	25
13.5. I. 网络底层与并发架构	26
13.6. II. 多智能体协作与大模型赋能	27
13.7. III. 全栈工程化与极致 UI	28
14. 实现状态标注	29
14.1. WebSocket 消息类型 (第 6 章)	29
14.2. 揭棋规则与领域层	30
14.3. TCP 扩展 (附录 B)	30
14.4. 工程与 DevOps	30
14.5. v2.0 内容保留检查清单 (§ 0.2 自检)	31
15. 附录 A: WS JSON messageType 快速参考卡片	32
16. 附录 B: TCP 文本帧扩展协议 v2.0 (Unveil 扩展)	32
16.1. B.1 消息帧格式	32

16.2.	B.2	消息帧解析规则	33
16.3.	B.3	消息类型目录	33
16.4.	B.4	各消息详细格式	34
16.4.1.	B.4.x	MSG_LOGIN (1) — 客户端登录	34
16.4.2.	B.4.x	MSG_MOVE (2) — 走子	35
16.4.3.	B.4.x	MSG_GAME_STATE (3) — 游戏状态	35
16.4.4.	B.4.x	MSG_ERROR (4) — 错误消息	36
16.4.5.	B.4.x	MSG_QUIT (5) — 退出	37
16.4.6.	B.4.x	MSG_GAME_OVER (6) — 游戏结束	37
16.4.7.	B.4.x	MSG_BOARD_STATE (7) — 棋盘同步	38
16.4.8.	B.4.x	MSG_DRAW_REQUEST (8) — 提和	39
16.4.9.	B.4.x	MSG_RESIGN (9) — 认输	40
16.4.10.	B.4.x	MSG_CHAT (10) — 聊天	40
16.5.	B.7	TCP 典型通信时序图	40
16.5.1.	B.7.1	正常对弈时序	41
16.5.2.	B.7.2	非法着法被拒绝	41
16.5.3.	B.7.3	超时判负	42
16.5.4.	B.7.4	提和流程	42
16.5.5.	B.7.5	认输流程	42
16.6.	B.8	TCP 消息快速参考卡片	42
17.		附录 C: 版本历史	43

§ = 节 (section)：正文章节引用，如 §6.5 即第 6 章第 5 节；附录 B §B.4 即附录 B 第 4 节。§0.2 仅指任务清单，非正文章节。

1. 基础约定

1.1. 术语定义

术语	含义
§ (节号)	章节引用符号；§ 6.5 = 第 6 章第 5 节。见目录后说明
暗子	背面朝上、尚未翻开的棋子，按所在位置对应的中国象棋棋子规则移动
明子	正面朝上、已翻开的棋子，按实际类型规则移动
翻子	将暗子变为明子的操作。可通过移动触发，也可原地翻子（消耗一回合）
先手	红方，行棋优先权，棋盘下方
后手	黑方，棋盘上方
回合	一方完成一次走子或翻子操作
半步	一方的一次走子（40 回合 = 双方共 80 个半步）

1.2. 技术约定

项目	约定	备注
传输协议	WebSocket + JSON	课程公共接口；默认端口 8887；见第 6 章
TCP 扩展	文本帧 <code>msgType len payload\n</code>	本组保留；默认端口 8888；见附录 B
字符编码	UTF-8	JSON 与 TCP payload 均为 UTF-8
TCP 行尾	LF (<code>\n</code> , 0x0A)	附录 B 每条 TCP 消息以单个换行符结尾
WS 消息识别	<code>messageType</code> 字符串	每条 JSON 对象必含此字段
心跳	<code>ping</code> / <code>pong</code> ，建议 10s	未实现心跳的客户端应被兼容
超时阈值	65 秒（60 秒思考 + 5 秒网络裕量）	服务器可配置

项目	约定	备注
时间戳单位	毫秒	<code>System.currentTimeMillis()</code> 风格
时间戳权威方	服务器	客户端时间戳仅供参考，超时判定以服务器为准

2. 坐标系统

2.1. 坐标定义

棋盘为 10 行 × 9 列，坐标采用“字母列 + 数字行”的字符串表示。

维度	范围	方向
行	0 - 9（共 10 行）	从上到下依次为 9, 8, ..., 1, 0
列	a - i（共 9 列）	从左到右依次为 a, b, ..., i

	a	b	c	d	e	f	g	h	i
9	車	馬	象	士	將	士	象	馬	車
8	•	•	•	•	•	•	•	•	•
7	•	炮	•	•	•	•	•	炮	•
6	卒	•	卒	•	卒	•	卒	•	卒
5	•	•	•	•	•	•	•	•	•
4	•	•	•	•	•	•	•	•	•
3	兵	•	兵	•	兵	•	兵	•	兵
2	•	炮	•	•	•	•	•	炮	•
1	•	•	•	•	•	•	•	•	•
0	車	馬	相	士	帥	士	相	馬	車

Table 1: 棋盘坐标表格（行 0 = 红方底线，行 9 = 黑方底线，粗线分隔楚河汉界）
重要约定：

- 先手（红方）始终在下方（行 0 - 4），后手（黑方）在上方（行 5 - 9）
- 客户端 UI 可将己方置于下方显示，但逻辑坐标必须基于上图坐标系
- 坐标字符串中行号直接使用显示行号，不翻转（如红方帅 = "e0"，黑方将 = "e9"）

2.2. 坐标转换公式

坐标字符串与内部棋盘数组索引的转换，本组提供参考：

```
// 坐标字符串 → 内部数组索引
// "b3" → row=6, col=1 (数组 row 0 = 棋盘顶行)
public static int[] fromCoord(String coord) {
    return new int[]{
        9 - (coord.charAt(1) - '0'), // 行：显示行号 → 数组索引
        coord.charAt(0) - 'a'        // 列：'a' → 0
    };
}

// 内部数组索引 → 坐标字符串
// row=6, col=1 → "b3"
public static String toCoord(int row, int col) {
    return "" + (char)('a' + col) + (9 - row);
}
```

3. 棋子类型编码

编码	红方名称	黑方名称	基准价值	备注
0	帅	将	10000	开局即明，双方各 1 枚
1	车	车	600	双方各 2 枚
2	马	马	270	蹩马腿规则同象棋
3	炮	炮	285	翻山吃子规则同象棋
4	兵	卒	30	过河后可横移，双方各 5 枚
5	仕	士	120	明子后可离宫、可过河，斜走一格
6	相	象	120	明子后可过河，塞象眼规则不变

注意：基准价值仅作为 AI 评估函数的参考，组间互操作不依赖此值。

3.1. 老师 JSON piece 枚举映射 (§ 3.2)

老师文档 piece 枚举首字母大写 (Rook/Knight/…), JSON 示例使用全小写。本组统一全小写 (PieceJsonMapper)，组间互操作以联调为准。

内部编码	中文名	JSON piece	红方	黑方	状态
0	将/帅	king	帅	将	已实现

内部编码	中文名	JSON piece	红方	黑方	状态
1	车	rook	车	車	已实现
2	马	knight	马	馬	已实现
3	炮	cannon	炮	砲	已实现
4	兵/卒	pawn	兵	卒	已实现
5	士/仕	guard	仕	士	已实现
6	象/相	bishop	相	象	已实现

3.2. 内部编码 ↔ JSON ↔ 中文三向对照 (§ 3.3)

内部编码	JSON piece	红方中文	黑方中文
0	king	帅	将
1	rook	车	车
2	knight	马	马
3	cannon	炮	炮
4	pawn	兵	卒
5	guard	仕	士
6	bishop	相	象

注意：内部编码 0 - 6 用于领域层、棋谱记法与附录 B BOARD_STATE；JSON 层仅使用 piece 英文字符串。

4. 颜色编码

编码	阵营	说明
0	红方	先手，棋盘下方
1	黑方	后手，棋盘上方

5. Move 对象规范

5.1. 类定义

```
public class Move {  
    private String source;           // 起点坐标, 如 "a0"  
    private String destination;     // 终点坐标, 如 "a1"  
    private Integer type;           // 翻出的棋子类型 (0-6), 非翻子步为 null  
    private long turnStartTime;     // 回合开始时间戳 (毫秒), 以服务器记录值  
    为准  
    private long clientTimestamp;   // 客户端发送时间戳 (服务器忽略防伪造)  
    private long serverTimestamp;   // 服务器处理时间戳 (服务器写入)  
    private boolean isFlipOnly;     // 是否原地翻子操作  
}
```

5.2. 字段规则

规则	说明
首翻必带 type	暗子首次被移动或翻开后, 服务器必须将真实 type 填入并广播
非首翻 type 为空	已知明子的移动, type 字段为 null 或留空
时间戳权威	超时判定以服务器回合开始时间为准, 客户端时间戳仅供参考
flipOnly 等价性	<code>source.equals(destination) ⇔ isFlipOnly == true</code> , 两者等效
翻子随机性	服务器在棋局初始化时完成暗子随机排列, 翻子时仅揭示预置类型; 客户端提交的 type 被忽略并覆盖

5.3. 翻子随机性机制

- 服务器在棋局初始化时完成暗子随机排列 (每方 15 枚暗子, 从类型池随机分配至各位置)
- 类型池 (每方): 车 ×2、马 ×2、炮 ×2、卒 ×5、士 ×2、象 ×2
- 客户端无法预知暗子真实类型
- 客户端走子/翻子时, 服务器仅揭示已预置的真实类型
- 翻子后该棋子类型永久固定

6. 安全性：客户端无法通过伪造 type 值改变翻子结果

5.4. JSON move 对象格式 (§ 5.4)

与老师文档一致，坐标拆分为列字母与行数字：

JSON 字段	类型	说明
fromX / toX	String	列 a - i
fromY / toY	int	行 0 - 9 (0=红方底线, 9=黑方底线)
isFlip	boolean	true=本步翻子; from==to 时为原地翻子

映射: "b"+1 → source="b1"; JsonMessages.parseMove / toMoveJson 负责与 Java Move 互转。

5.5. JSON moveResult 对象 (§ 5.5)

字段	类型	说明	状态
success	boolean	消息处理是否成功	已实现
valid	boolean	走子是否符合规则	已实现
move	object	同客户端 move 结构	已实现
flipResult	String?	isFlip 为真时, 翻出棋子 piece 枚举	已实现

完整 WS 消息定义见第 6 章。

6. WebSocket + JSON 通信协议

6.1. 传输层约定 (§ 6.1)

客户端与服务器通过 WebSocket 交换 JSON 文本。每条消息为一个 JSON 对象，必须包含 messageType 字段：

```
{"messageType": "move", "fromX": "b", "fromY": 1, "toX": "b", "toY": 3, "isFlip": false}
```

项目	约定	说明	状态
传输	WebSocket 文本帧	一帧 = 一条 JSON 字符串	已实现

项目	约定	说明	状态
连接 URL	ws://host:8887	默认端口 8887；启动时打印	已实现
编码	UTF-8	JSON 字符串	已实现
心跳	可选，建议 10s ping/pong	未实现心跳的客户端应被兼容	已实现
未知 messageType	静默忽略，不得崩溃	老师原文；本组返回 error 4001	部分实现

实现类：com.jieqi.protocol.json.JsonMessages、
com.jieqi.server.ws.WsGameServer、com.jieqi.client.WsGameClient。

6.2. 依赖库版本（§ 6.2）

老师示例指定，各组统一：

依赖	版本	状态
org.java-websocket:Java-WebSocket	1.5.7	已实现
com.google.code.gson:gson	2.10.1	已实现

6.3. 消息格式总览（§ 6.3）

所有消息必含 messageType（字符串）。方向缩写：C→S = 客户端→服务器，S→C = 服务器→客户端。

6.4. 客户端 → 服务器消息（§ 6.4）

messageType	方向	说明	字段	状态
Login	C→S	登录	userId, password	已实现
register	C→S	注册	userId, password, nickname	已实现
startMatch	C→S	开始匹配	无额外字段	已实现
cancelMatch	C→S	取消匹配	无额外字段	已实现
requestFirstHand	C→S	请求先手（10s 窗口）	wannaFirst: true/false	已实现
Ready	C→S	准备就绪	无额外字段	已实现

messageType	方向	说明	字段	状态
move	C→S	走子/翻子	fromX, fromY, toX, toY, isFlip	已实现
ping	C→S	心跳	timestamp: long 毫秒	已实现
Resign	C→S	认输	无额外字段	已实现

字段类型 (C→S) :

字段	类型	必填	说明
userId / password / nickname	String	是	账号信息
wannaFirst	boolean	是	true=请求红方先手
fromX / toX	String	是	列 a - i
fromY / toY	int	是	行 0 - 9
isFlip	boolean	是	true=本步翻子; from=to 时为原地翻子
timestamp	long	是	毫秒时间戳, pong 原样返回

JSON 示例 (C→S) :

```
Login
{
  "messageType": "Login",
  "userId": "u1",
  "password": "123456"
}
```

```
register
{
  "messageType": "register",
  "userId": "u2",
  "password": "123456",
  "nickname": "李四"
}
```

```
startMatch / cancelMatch / Ready / Resign
{
  "messageType": "startMatch"
}
```

```
{
  "messageType": "cancelMatch"
}
{
  "messageType": "Ready"
}
{
  "messageType": "Resign"
}
```

requestFirstHand

```
{
  "messageType": "requestFirstHand",
  "wannaFirst": true
}
```

move (含翻子)

```
{
  "messageType": "move",
  "fromX": "a",
  "fromY": 0,
  "toX": "a",
  "toY": 1,
  "isFlip": true
}
```

ping

```
{
  "messageType": "ping",
  "timestamp": 1712345678901
}
```

6.5. 服务器 → 客户端消息 (§ 6.5)

messageType	方向	说明	字段	状态
loginResult	S→C	登录结果	success, message, userId (成功时)	已实现
matchSuccess	S→C	匹配成功	roomId, opponentId, opponentNickname	已实现
roomInfo	S→C	房间状态	opponentReady: true/false	已实现
gameStart	S→C	开局	redPlayerId, blackPlayerId, yourColor, firstHand, initialBoard	已实现

messageType	方向	说明	字段	状态
moveResult	S→C	走子结果	success, valid, move, flipResult?	已实现
timeout	S→C	超时判负	loserId, winnerId, reason	已实现
gameOver	S→C	终局	winner, reason, winnerId	已实现
pong	S→C	心跳回复	timestamp	已实现
error	S→C	错误	code, message	已实现

走子结果广播规则（老师原文）：`valid=true` 时向双方广播；`valid=false` 时仅回复发送方。

JSON 示例（S→C）：以下每条消息单独成框；`raw` 不换行会导致长 JSON 撑破灰框，故采用可换行等宽文本。

```
loginResult
{
  "messageType": "loginResult",
  "success": true,
  "message": "ok",
  "userId": "u1"
}
```

```
matchSuccess
{
  "messageType": "matchSuccess",
  "roomId": "room_123",
  "opponentId": "user456",
  "opponentNickname": "象棋高手"
}
```

```
roomInfo
{
  "messageType": "roomInfo",
  "opponentReady": true
}
```

```
gameStart
{
  "messageType": "gameStart",
  "redPlayerId": "user123",
  "blackPlayerId": "user456",
  "yourColor": "red",
  "firstHand": true,
  "initialBoard": [
    {
```

```
"x": "a",
"y": 0,
"piece": "rook",
"visible": false
}
]
```

moveResult (valid=true, 双方广播)

```
{
  "messageType": "moveResult",
  "success": true,
  "valid": true,
  "move": {
    "fromX": "a",
    "fromY": 0,
    "toX": "a",
    "toY": 1,
    "isFlip": true
  },
  "flipResult": "cannon"
}
```

timeout

```
{
  "messageType": "timeout",
  "loserId": "user123",
  "winnerId": "user456",
  "reason": "timeout"
}
```

gameOver

```
{
  "messageType": "gameOver",
  "winner": "red",
  "reason": "checkmate",
  "winnerId": "user123"
}
```

pong

```
{
  "messageType": "pong",
  "timestamp": 1712345678901
}
```

error

```
{
  "messageType": "error",
  "code": 2001,
}
```

```
"message": "非法走子"
}
```

6.6. 公共数据结构（§ 6.6）

6.6.1. initialBoard

棋盘为 9×10，每格一个对象（仅包含有子格子）：

```
{
  "x": "a",
  "y": 0,
  "piece": "rook",
  "visible": false
}
```

字段	类型	取值	状态
x	String	a - i	已实现
y	int	0 - 9（0=红方底线）	已实现
piece	String	king/rook/... 见 § 3.2	已实现
visible	boolean	false=暗子，true=明子	已实现

- 开局除将帅外 `visible=false`；翻子后服务器写入真实 `piece` 并设 `visible=true`。
- `flipResult` 使用相同英文枚举（如 "cannon"）。
- 服务器权威：客户端不可伪造翻子结果；`RandomRevealService` 在走子后写回。

6.6.2. move 对象（JSON）

字段	类型	说明
fromX / toX	String	列 a - i
fromY / toY	int	行 0 - 9
isFlip	boolean	true=翻子步；from=to 时为原地翻子

内部领域对象使用 `Move.source` / `Move.destination` 合并字符串（如 "b1"），边界处由 `JsonMessages` 转换。

6.6.3. moveResult 对象

字段	类型	说明	状态
success	boolean	消息处理是否成功	已实现
valid	boolean	走子是否符合规则	已实现
move	object	同客户端 <code>move</code> 结构	已实现
flipResult	String?	isFlip 为真时，翻出棋子 <code>piece</code> 枚举	已实现

6.7. 错误码（§ 6.7）

code	含义（老师原文）	状态
1001	登录失败（账号或密码错误）	已实现
1002	重复登录	已实现
2001	非法走子（规则不符）	已实现
2002	未轮到本方走子	已实现
2003	超时未走子	已实现
3001	房间不存在	已实现
3002	匹配失败（无对手）	已实现
4001	JSON 格式错误	已实现

6.7.1. gameOver / timeout 原因（reason）

reason	说明	状态
checkmate	将死（老师原文）	已实现
resign	认输（老师原文）	已实现
timeout	超时	已实现
stalemate	困毙	已实现
disconnect	断线判负	已实现
king_captured	吃将获胜（允许不应将）	已实现
draw_no_capture	40 回合无吃子和	已实现
repetition_loss	长将/长捉判负	已实现
repetition_draw	兵卒长捉和	已实现
draw_agreed	协议和棋	已实现

winner 取值: "red" / "black"; 和棋时本组扩展使用 "draw"。

6.8. 典型 JSON 示例

以下示例对应老师公共接口原文 § 4.1 – § 4.5。

6.8.1. 匹配与先手协商

```
// C→S 开始匹配
{"messageType": "startMatch"}
```

```

// S→C 匹配成功
{
  "messageType": "matchSuccess",
  "roomId": "room_123",
  "opponentId": "user456",
  "opponentNickname": "象棋高手"
}
// C→S 请求先手
{
  "messageType": "requestFirstHand",
  "wannaFirst": true
}
// S→C 游戏开始
{
  "messageType": "gameStart",
  "redPlayerId": "user123",
  "blackPlayerId": "user456",
  "yourColor": "red",
  "firstHand": true,
  "initialBoard": [
    {
      "x": "a",
      "y": 0,
      "piece": "rook",
      "visible": false
    }
  ]
}

```

6.8.2. 走子与翻子

```

// C→S 移动 (翻子)
{
  "messageType": "move",
  "fromX": "a",
  "fromY": 0,
  "toX": "a",
  "toY": 1,
  "isFlip": true
}
// S→C 移动结果 (广播)
{
  "messageType": "moveResult",
  "success": true,
  "move": {
    "fromX": "a",
    "fromY": 0,
    "toX": "a",
    "toY": 1,
    "isFlip": true
  },
  "flipResult": "cannon",
  "valid": true
}

```

6.8.3. 超时处理

```
// S→C 超时 (广播)
{
  "messageType": "timeout",
  "loserId": "user123",
  "winnerId": "user456",
  "reason": "timeout"
}
```

6.8.4. 正常结束

```
// S→C 将死对方
{
  "messageType": "gameOver",
  "winner": "red",
  "reason": "checkmate",
  "winnerId": "user123"
}
```

6.8.5. 心跳

```
// C→S
{
  "messageType": "ping",
  "timestamp": 1712345678901
}
// S→C
{
  "messageType": "pong",
  "timestamp": 1712345678901
}
```

6.9. 本组扩展消息

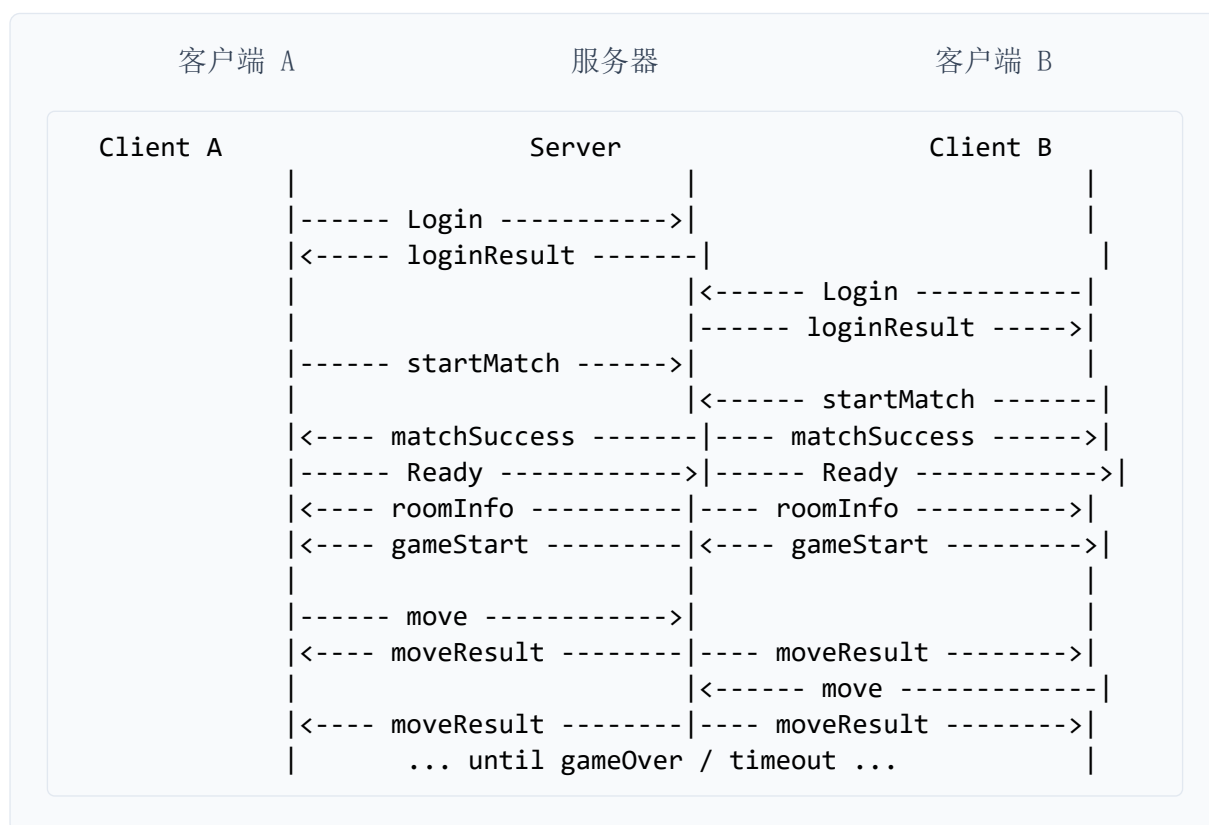
在不影响与老师客户端互操作的前提下，本组保留以下扩展能力（见附录 B TCP 层）：

- 提和 / 聊天 / 多盘 `gameId` — 仅 TCP 扩展
- 步时 65s、长将/长捉、吃将获胜等规则扩展在服务器逻辑中实现，通过扩展 `gameOver.reason` 表达

7. 典型通信流程

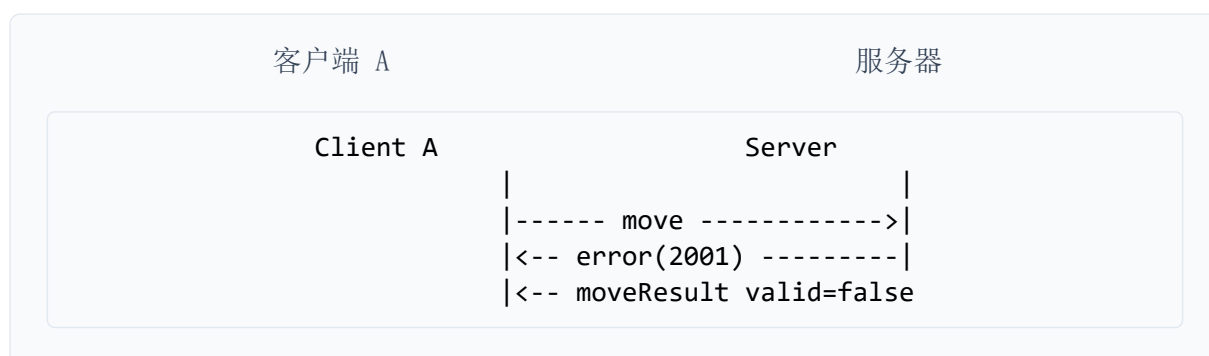
时序图内为 JSON `messageType` 标签；WebSocket 全双工。

7.1. 正常对弈时序



Listing 1: WebSocket JSON: 登录 → 匹配 → Ready → 开局 → 走子

7.2. 非法着法



Listing 2: 非法着法: 仅发送方收到 error + valid=false 的 moveResult

7.3. 超时判负



Listing 3: 超时: 广播 timeout + gameOver (reason=timeout)

7.4. 认输



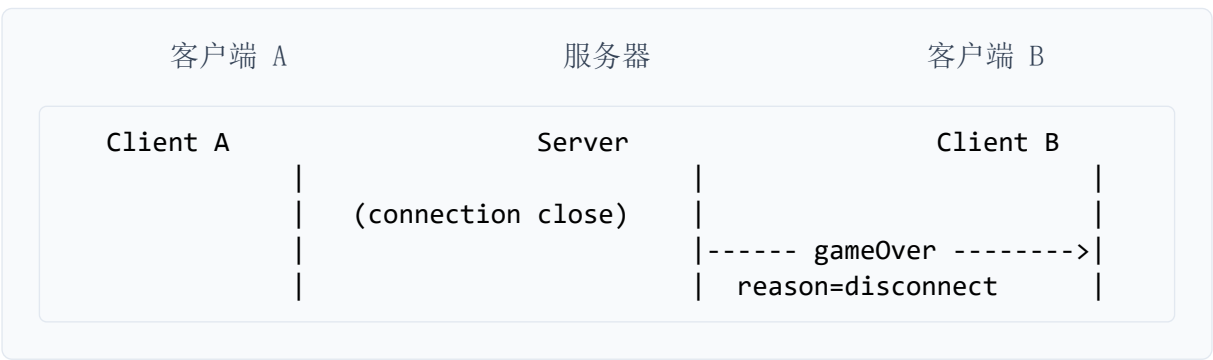
Listing 4: 认输: 广播 gameOver (reason=resign)

7.5. 先手协商时序 (requestFirstHand)



Listing 5: 可选: 10s 窗口内双方发送 requestFirstHand, 服务器分配红/黑

7.6. 断线判负



Listing 6: 一方 WebSocket 关闭：对方收到 gameOver (reason=disconnect)

注：附录 B §B.7 保留 v2.0 TCP 文本帧典型时序图（LOGIN/MOVE/BOARD_STATE 等），供 TCP 联调参考。

8. 暗子走法与虚拟类型

8.1. 虚拟类型机制

暗子未翻开时，其移动规则由所在位置的“虚拟类型”决定，而非实际类型。
虚拟类型 = 该位置按中国象棋初始布局本应放置的棋子类型。

	a	b	c	d	e	f	g	h	i
9	車	馬	象	士	將	士	象	馬	車
8	—	—	—	—	—	—	—	—	—
7	—	炮	—	—	—	—	—	炮	—
6	卒	—	卒	—	卒	—	卒	—	卒
5	—	—	—	—	—	—	—	—	—
4	—	—	—	—	—	—	—	—	—
3	兵	—	兵	—	兵	—	兵	—	兵
2	—	炮	—	—	—	—	—	炮	—
1	—	—	—	—	—	—	—	—	—
0	車	馬	相	士	帥	士	相	馬	車

Table 2: 棋盘各位置对应的虚拟类型（— 表示无棋子，红底格为开局即明的将/帅）
说明：

- 将帅位置（0e 和 9e）为明子，实际类型 = 虚拟类型 = 将/帅

- 暗子严格按虚拟类型的中国象棋原始规则移动（含位置限制），不享有明子强化
- 例如：位于士位的暗子只能斜走一格于九宫内；翻开后若为明子士，方可离宫过河

8.2. 明子强化规则

暗子翻开为明子后，士和象获得强化：

棋子	暗子状态	明子状态
士 / 仕	斜走一格，限于九宫内	斜走一格，可离宫、可过河
象 / 相	田字走法，不可过河，塞象眼有效	田字走法，可过河，塞象眼不变

其他棋子的走法规则与中国象棋完全一致。

9. 胜负与和棋判定

9.1. 胜负条件

条件	结果	说明
将死	对方胜	被将军且无任何合法着法可解除将军
困毙	对方胜	未被将军但没有任何合法着法可走
超时	对方胜	单步超过 65 秒未走子
认输	对方胜	主动发送 RESIGN 消息
断线	对方胜	TCP 连接异常断开
不应将	对方下一步吃将后胜	系统不自动判负，由对方通过正常吃子操作实现

9.2. 和棋条件

条件	说明
40 回合无吃子	双方连续 80 个半步无任何吃子发生（翻子不算吃子）
协议和棋	一方提和，对方同意
兵卒长捉	兵/卒连续长捉导致同局面重复 ≥ 6 次，判和而非判负

9.3. 长将 / 长捉判定

类型	判定
长将	同一方连续将军 ≥ 6 次，且局面重复（哈希相同），判负（对方胜）
长捉（非兵卒）	同一方连续捉子 ≥ 6 次，且局面重复，判负（对方胜）
兵卒长捉	兵/卒连续捉子导致局面重复 ≥ 6 次，判和

实现细节：

- 局面重复判定使用局面哈希（含每个位置的明子类型/颜色/暗子标记 + 当前走子方）
- 暗子的虚拟类型不参与哈希（同位置虚拟类型固定）
- 计数器在每次吃子后重置

9.4. 超时判定公式

```
if (serverCurrentTime - serverTurnStartTime > 60000 + 5000) {  
    // 当前走子方超时判负  
}
```

- 60000 ms = 60 秒（每步时限）
- 5000 ms = 5 秒（网络延迟裕量）
- 总阈值 = 65000 毫秒

注意：

- 回合开始时间 `serverTurnStartTime` 由服务器在每次切换走子方时记录
- 客户端 Move 中的 `turnStartTime` 不被信任，仅用于日志
- 若客户端发送 MOVE 时已超时，服务器拒绝走子并判当前方负

10. 棋谱记录格式

10.1. 每步棋的记法

```
<步数> . <源坐标>-<目标坐标>[( <翻出棋子类型> )] [翻]
```

示例：

1. b1-c3(2) 第 1 步, 走子 b1→c3, 翻出类型 2 (马)
2. h7-g7 第 2 步, h7→g7 (明子移动, 无翻子)
3. a0-a0(1)[翻] 第 3 步, 原地翻开 a0, 翻出类型 1 (车)

10.2. 棋子类型的棋谱名称

类型编码	红方名称	黑方名称
0	帅	将
1	车	车
2	马	马
3	炮	炮
4	兵	卒
5	仕	士
6	相	象

10.3. 服务器棋谱存储

服务器必须按时间顺序保存每局棋的所有走法。建议格式:

- 内存: `List<Move>` 或 `List<String>`, 实时追记
- 持久化: 对局结束时写入文件, 文件名为 "`<gameId>_<时间戳>.pgn`" 或 ".txt"
- 棋谱内容应包含对局元信息 (双方昵称、起始时间、终局原因)

11. 多盘对弈

支持一台服务器同时进行多局对弈。

11.1. 实现要点

1. 每局棋分配唯一 `gameId` (建议 UUID 前 8 位) 客户端 LOGIN 时通过 `gameId` 字段指定加入目标对局 `gameId` 为空时服务器自动匹配 服务器内部以 `Map<String, Game>` 管理多局 (线程安全集合)

11.2. 跨组兼容

- 若对方服务器支持多盘: 正常指定 `gameId` 加入

- 若对方服务器不支持多盘（仅单局）：拒绝 `gameId` 非空的 LOGIN，返回 ERROR 200（游戏房间不存在）
- 客户端应对此错误做友好提示

12. 组间联调检查清单

以下条目为组间联调准入标准，所有接入方（含本组）在联调前均应逐项自检通过：

1. 消息帧格式正确：`msgType|payloadLen|payload\n`，UTF-8 编码
2. 收到未知 `msgType`（8/9/10/100+）不会崩溃，静默忽略
3. 坐标解析正确：`"a0"` = 左下角（红方底线左车位），`"i9"` = 右上角（黑方底线右车位）
4. 翻子操作 `source == destination` 被正确处理为原地翻子
5. 服务器广播的 MOVE 中 `type` 为服务器生成值，非客户端提交值
6. BOARD_STATE 的 `row0` = 棋盘最顶行（黑方），`row9` = 最底行（红方）
7. BOARD_STATE 能完整解析并重建棋盘对象
8. GAME_OVER 的 `winner == -1` 时正确显示“和棋”
9. GAME_OVER 正确携带 `reasonCode` 和 `reasonDescription`
10. GAME_STATE 的子类型（LOGIN_ACK / GAME_START / TURN_CHANGE）均正确解析
11. 超时默认 65s，使用服务器时间戳判定
12. 端口号在启动时明确打印到控制台
13. 错误消息（MSG_ERROR）能解析错误码并友好展示
14. 对方断线时本方收到 GAME_OVER（原因码 = 4）而非无限等待

13. 待老师确认的开放问题

作业题目说明「本题目定义应该是不完整的」。本节为本组在需求分析、规则研读、协议设计与联调中 主动提出并给出暂定方案 的争议点，供老师裁定后写入全体统一的公共规范。

说明：带「待确认」者需教师裁定；带「本组方案」者为本协议 v3.0 已采用、建议老师认可后全组遵照的默认实现。

13.1. 规则与胜负判定

序号	问题	本组暂定方案	状态
Q1	吃暗子时，被吃方是否应知道被吃子的真实类型？规则写「被吃方不知道」，但 MOVE 广播含 type。	方案 A：双方广播相同（实现简单）； 方案 B：对被吃方发送 type 置空的 MOVE（与规则一致，需双通道广播）。	待确认
Q2	题目写「不考虑不应将」，走子后己方被将军是否仍一律拒绝？若对方未应将，是否允许直接吃帅结束？	允许送将：仅 isValidMove 校验；不因己方被将拒绝。对方下一步吃明将/帅 → KING_CAPTURED (5)。见正文「不与不应将自动判负」。	本组方案
Q3	「40 回合无吃子」指 40 个完整回合（80 半步）还是 40 半步？与实现 noCaptureCount >= 80 是否一致？	采用 80 半步（等价双方各 40 步无吃子），GAME_OVER 原因码 6。	待确认
Q4	长将/长捉「6 次」是否必须连续？中间插入非将军、非捉子步是否重置计数？	同局面哈希累计达 6 次判负/和（不强制连续）；兵卒长捉单独判和（Q5）。	待确认
Q5	「兵卒长捉判和」：兵卒长捉兵卒和，还是兵卒长捉任意子均和？	兵卒长捉导致重复局面 ≥6 次判和（不限被捉子类型）。	待确认
Q6	困毙、无合法走子（含仅能原地翻子）是否均判负？单方只剩将帅是否和棋？	无合法着法（含翻子）且未被将时判困毙负；单方将帅判和（实现可扩展）。	待确认
Q7	将帅是否允许照面（中间无子）？亚洲规则常见「不可照面」。	允许照面；照面时不判负（与中国象棋部分规则一致，待权威裁定）。	待确认
Q8	暗子在士/象位：是否享受明士/明象的过河强化？还是严格按位置虚拟类型的原始象棋规则？	否：暗子按虚拟类型走子（士限九宫、象不过河）；翻开为明后才强化。	本组方案

13.2. 翻子、随机与棋谱

序号	问题	本组暂定方案	状态
Q9	暗子首次翻开时 type 由谁生成？客户端上传的 type 是否一律由服务器覆盖？	仅服务器随机生成并写入广播；忽略客户端 type（防作弊）。	本组方案
Q10	随机打乱是否需可复现（公开 random seed）以便复盘与 AI 调试？	联调不要求；棋谱记录翻出后的 type；可选日志记录 seed（扩展）。	待确认
Q11	原地翻子（source=destination）是否每回合仅限一枚？能否连续多回合只翻子？	每回合仅允许一次翻子或走子；可连续多回合只翻子（合法即允许）。	待确认
Q12	棋谱中首翻是否必须记录 type？后续明子走子 type 字段是否应为空？	首次翻开必记 type；后续步 type 为空字符串；原地翻子标记 [翻]。	本组方案
Q13	吃子后翻开的 type 是否对吃子方立即可见、对被吃方是否隐藏（见 Q1）？	与 Q1 联动；当前协议对双方广播相同 MOVE。	待确认

13.3. 网络、计时与互操作

序号	问题	本组暂定方案	状态
Q14	每步限时：60 秒 + 5 秒网络裕量是否为课程统一标准？AI 本地对弈是否同标准？	网络对战 65s（服务器 turnStartTime）；AI 本地可自定，报告说明即可。	待确认
Q15	客户端 turnStartTime 是否完全忽略？是否保留用于日志与排错？	判超时仅以服务器时间为准；客户端时间戳可记录但不参与判定。	本组方案
Q16	TCP 端口是否统一？不同组默认端口不一致时如何联调？	建议统一默认 8888；各组可改端口但启动时打印，文档写明。	本组

序号	问题	本组暂定方案	状态
			方案
Q17	断线重连：掉线后能否用相同身份重连并恢复棋局？是否纳入公共协议？	v2.0 未定义；断线判对方胜（原因码 4）；重连列为 v2.1 扩展建议。	待确认
Q18	多盘对弈： <code>gameId</code> 为空时自动匹配规则？满员后是否新建房间？	先匹配 WAITING 房间；无则创建 UUID 前 8 位；满员拒绝（ERROR 201）。	本组方案
Q19	观战/裁判：第三方只读客户端是否需标准消息（如 SPECTATOR）？	暂不纳入必做；观战组可订阅 BOARD_STATE 扩展实现。	待确认
Q20	和棋：提和是否需双方 OFFER 后 ACCEPT？单方提和是否足够？	一方 ACCEPT 即和（原因码 9）；拒绝则 DECLINE 继续。	本组方案

13.4. AI 博弈与评估（选做 AI 部分的组可一并裁定）

序号	问题	本组暂定方案	状态
Q21	暗子评估：未翻开子用虚拟位置价值还是剩余池期望值？是否向老师报备评估公式？	互操作不传输评估；各组 AI 自定，实验报告公开公式与胜负统计。	本组方案
Q22	AI 与真人对弈是否必须连接本协议服务器？能否本地共用一个 Board 类？	鼓励共用 <code>jieqi-core</code> 领域模块；网络 AI 作特殊客户端连接服务器。	待确认
Q23	Agent 对象是否需统一接口（如 <code>JieqiAgent.selectMove(Board)</code> ）以便组间 AI 对战？	不强制；建议课程提供可选接口约定，便于擂台赛。	待确认

序号	问题	本组暂定方案	状态
Q24	随机翻子导致搜索树膨胀：AI 是否允许「期望着法」与「实际翻开」分支不一致的日志说明？	允许；报告记录期望分数与实际结果的差异样本。	待确认

以下 Q25 - Q44 为本组提出的 扩展方向（非作业必做），用于完善题目边界、争取课程加分，并供老师裁定是否纳入公共规范 v2.1 或仅作本组实验报告亮点。

方向	编号
I 网络底层与并发	Q25 - Q32
II 多智能体与 LLM	Q33 - Q39
III 全栈工程化与 UI	Q40 - Q44

13.5. I. 网络底层与并发架构

序号	问题	本组暂定方案	状态
Q25	TCP 粘包/半包：仅用 <code>readLine()</code> 按 <code>\n</code> 切帧是否足够？是否应强制「长度字段 + 字节缓冲」解码？	v2.0 帧格式为 <code>msgType\ payloadLen\ payload\n;</code> 推荐实现 <code>FrameDecoder</code> （环形缓冲/ByteBuffer），先凑满 <code>payloadLen</code> 再解析， <code>\n</code> 仅作帧尾校验； 报告专节说明粘包/半包与半包恢复流程。	本组方案
Q26	<code>payloadLen</code> 与 UTF-8 实际字节数不一致时如何处理？多字节中文 CHAT 是否计入长度？	长度按 UTF-8 字节计；不一致则 ERROR 111 并丢弃该帧剩余字节（防协议错乱）。	本组方案
Q27	单帧 <code>payloadLen</code> 上限是否课程统一？防止恶意客户端撑爆内存。	建议上限 64 KiB；超限关闭连接并记日志（待老师确认数值）。	待确认
Q28	高并发：每连接一线程（BIO）还是 <code>NIO Selector</code> / 虚拟线程？是否影响互操作？	互操作仅约束字节流语义，不约束 IO 模型；本组 server 可迭代为 <code>NIO</code> ，报告对比吞吐。	待确认

序号	问题	本组暂定方案	状态
Q29	多盘对弈规模扩大后，内存 <code>Map<gameId, GameSession></code> 是否改为 Redis 缓存对局状态？	必做可仍用内存；加分路径： <code>jieqi-server</code> + Redis 存 <code>BOARD/</code> <code>MOVE</code> 序列与房间元数据； 键命名建议 <code>jieqi:room:</code> <code>{gameId}</code> 、 <code>jieqi:queue:waiting</code> 。	待确认
Q30	Redis 匹配池 (Matchmaking Queue)：LOGIN 时 <code>gameId</code> 为空是否先入队再 pop 配对？	与 Q18 一致逻辑，队列可用 Redis LIST；无 Redis 时退化为内存 Map 扫描。	待确认
Q31	分布式下计时器：超时判定是否仍以游戏进程本地时钟为准，还是 Redis TTL / 独立调度服务？	联调最小方案：仍以持局进程 <code>turnStartTime</code> 为准；Redis 仅缓存状态不替代判时。	待确认
Q32	容器化交付：Dockerfile + <code>docker-compose.yml</code> 是否作为推荐提交物（含 server、可选 Redis）？	提供一键 <code>docker compose up</code> ；默认暴露 8888；环境变量 <code>JIEQI_PORT</code> 、 <code>REDIS_URL</code> ；不强制他组使用，本组实验报告附部署节。	本组方案

13.6. II. 多智能体协作与大模型赋能

序号	问题	本组暂定方案	状态
Q33	AI 是否必须从单体类拆为多 Agent：SearchAgent、ProbabilityAgent、EndgameAgent？	作业仅要求「至少一个 Agent 对象」；本组建议拆分并 <code>Orchestrator</code> 编排，报告附架构图； 不强制他组拆分。	待确认
Q34	多 Agent 决策合并：串行（先概率修正评估再搜索）还是并行投票？	本组采用串行管道：Probability → Search；残局库命中则短路返回。	本组方案
Q35	是否定义可选接口 <code>JieqiSubAgent.contribute(Board, Context)</code> 便于组间 AI 模块互换？	课程级可选约定，置于 <code>jieqi-ai</code> ；互操作仍只依赖 TCP MOVE。	待确认

序号	问题	本组暂定方案	状态
Q36	协议已预留 MSG_CHAT(10)：是否允许 AI/LLM 自动发「心理战/解说」？是否算正式功能？	允许作演示与加分；默认关闭；开启需在 LOGIN 或配置中声明 <code>allowChatAI=true</code> 。	待确认
Q37	LLM Prompt 输入范围：仅抽象局面（子力差、回合、是否被将）还是含完整 FEN/BOARD_STATE？	禁止上传对手隐私；仅发送脱敏摘要 + 己方视角；API Key 仅存服务端/本地配置。	本组方案
Q38	CHAT 频率与长度：是否限制（如每 30s 一条、 ≤ 200 字）？是否需敏感词过滤？	服务器限速 1 条/10s/人；超长截断；课程演示可人工审核开关。	待确认
Q39	LLM 失败（超时/配额）时：静默跳过还是发送固定 fallback 文案？	静默跳过，不影响对弈；CHAT 与 MOVE 解耦。	本组方案

13.7. III. 全栈工程化与极致 UI

序号	问题	本组暂定方案	状态
Q40	旁观/数据端：是否新增标准消息（如 STATS、SPECTATOR）广播胜率、暗子池概率？	v2.1 扩展；v2.0 观战可仅收 BOARD_STATE + 本地估算；STATS 字段格式待老师确认后写入 §4。	待确认
Q41	Bento 数据看板：胜率曲线、暗子雷达图由谁计算？服务器统一还是观战 Web 自算？	加分演示：观战端本地基于 BOARD_STATE + 公开评估公式；避免增加 server 负担。	本组方案
Q42	Web 旁观端（Vite/React 等）与 Console 客户端是否均需遵守同一 TCP 协议互操作？	是；Web 通过网关或 Java 后端代理 TCP，不另搞私有 WebSocket 除非另立附录协议。	待确认
Q43	「客户端不必过分美化」：Console 与 Bento Web 是否并存？评分是否只看功能正确？	Console 满足必做；Web 看板为加分展示，不替代走子客户端。	本组方案

序号	问题	本组暂定方案	状态
Q44	<code>docker-compose</code> 是否包含可选 Web 监控服务（如 :3000 旁观）与 Redis?	本组 <code>compose</code> 三服务： <code>server</code> 、 <code>redis</code> （profile 可选）、 <code>spectator-web</code> （profile 可选）。	本组方案

[Q45], [WS 与 TCP 双协议并存期间, 组间联调优先级如何确定?], [本组方案: 默认 WS 8887 联调; TCP 8888 作为备用/调试通道 (附录 B)。], [本组方案], 提交建议: 请老师对 Q1 - Q45 逐条确认或修正; Q25 - Q44 可整体标注为「加分扩展 / v3.1 草案」。确认后本组更新协议版本并通知他组。必做互操作以课程 WebSocket JSON 正文 + Q1 - Q24 规则扩展为准。

14. 实现状态标注

本章按本组实际代码与测试覆盖, 采用以下状态标注: **已实现** (代码完整且测试通过); **部分实现** (已实现, 但与规范存在差异或测试未全覆盖); **未实现** (尚未开发或仅作规划)。

14.1. WebSocket 消息类型 (第 6 章)

messageType / 特性	状态	说明
Login / register	已实现	WsGameServerIntegrationTest
startMatch / cancelMatch	已实现	含对手 error 3002 通知
requestFirstHand	已实现	10s 窗口 + 换色
Ready / roomInfo	已实现	
move (含 isFlip)	已实现	JsonMessages.parseMove 修复
loginResult / matchSuccess / gameStart	已实现	含 initialBoard
moveResult / flipResult	已实现	服务器权威翻子
timeout / gameOver	已实现	65s 阈值
ping / pong	已实现	
Resign	已实现	
error 1001 - 4001	已实现	12 场景集成测试

messageType / 特性	状态	说明
未知 messageType 静默忽略	部分实现	本组返回 error 4001

14.2. 揭棋规则与领域层

规则/模块	状态	说明
RandomRevealService 翻子随机	已实现	
RuleValidator 走法/虚拟类型	已实现	
EndgameJudge 将死/困毙/和棋	已实现	
长将/长捉/40 回合和棋	已实现	
暗子被吃信息差	部分实现	见 Q1; 双方广播相同 flipResult
不应将 (允许送将)	已实现	见 § 9
棋谱 .jieqi 存储	已实现	GameRecordStore
多盘并发 Map<String, Game>	已实现	

14.3. TCP 扩展 (附录 B)

特性	状态	说明
MSG 1 - 7 核心消息	已实现	端口 8888
MSG 8 - 10 扩展消息	已实现	DRAW/RESIGN/CHAT
BOARD_STATE Cell 编码	已实现	
FrameDecoder 帧解析	已实现	

14.4. 工程与 DevOps

特性	状态	说明
WsGameServerIntegrationTest 12 场景	已实现	jieqi-server
WsAIGameClient 自动对弈	已实现	jieqi-app 菜单 9 / ai-ws
Docker 默认 WS 8887	已实现	docker-compose.yml
Bento Web 旁观端	未实现	加分扩展 Q40 - Q44

特性	状态	说明
Redis 匹配队列	未实现	加分扩展 Q29 - Q32

14.5. v2.0 内容保留检查清单（§ 0.2 自检）

v2.0 内容	新文档位置	状态
术语定义表	§ 1.1	已实现
技术约定	§ 1.2 + 附录 B	已实现
坐标定义与转换公式	§ 2.1 - 2.2	已实现
棋子类型编码（含基准价值）	§ 3.1	已实现
颜色编码	§ 4	已实现
Move 类/字段/翻子随机性	§ 5.1 - 5.3	已实现
TCP 帧格式与解析	附录 B § B.1 - B.2	已实现
MSG 1 - 10 详细格式	附录 B § B.4	已实现
BOARD_STATE Ce11 与开局示例	附录 B § B.4.7	已实现
ERROR 100 - 202 / GAME_OVER 0 - 9	附录 B § B.5 - B.6	已实现
TCP 五组时序图	附录 B § B.7	已实现
虚拟类型/明子强化	§ 8.1 - 8.2	已实现
胜负/和棋/长将长捉/超时公式	§ 9.1 - 9.4	已实现
棋谱记录格式	§ 10	已实现
多盘对弈	§ 11	已实现
组间联调清单 14 条 + WS 扩展	§ 12	已实现
Q1 - Q44 开放问题	§ 13	已实现
附录 A TCP 速查	附录 B § B.8	已实现
版本历史 v0.0.0 - v2.0	附录 C	已实现
老师 WS+JSON 协议	§ 6 新增	已实现
实现状态标注	§ 14 新增	已实现

15. 附录 A: WS JSON messageType 快速参考卡片

messageType	方向	关键字段
Login	C→S	userId, password
register	C→S	userId, password, nickname
startMatch	C→S	—
Ready	C→S	—
move	C→S	fromX, fromY, toX, toY, isFlip
loginResult	S→C	success, message, userId
matchSuccess	S→C	roomId, opponentId, opponentNickname
gameStart	S→C	yourColor, firstHand, initialBoard
moveResult	S→C	valid, move, flipResult?
timeout	S→C	loserId, winnerId
gameOver	S→C	winner, reason, winnerId
error	S→C	code, message
ping / pong	双向	timestamp

本附录为 Unveil 历史扩展协议 (v2.0 正文第 6 章完整迁移), 供需要 TCP 通信的组参考或 telnet 调试使用。组间联调默认使用正文 WebSocket + JSON 协议 (第 6 章)。

Unveil 保留 TCP `msgType|payloadLen|payload\n` 实现 (端口 8888)。实现类:

`Protocol.java`、`GameServer`、`GameClient`、`FrameDecoder`。

组间互操作: 与对方联调前须约定使用 WebSocket JSON 或 TCP v2.0 (本附录); 不可混用。

16. 附录 B: TCP 文本帧扩展协议 v2.0 (Unveil 扩展)

16.1. B.1 消息帧格式

每条消息为一行文本 (以 `\n` 结尾):

```
<msgType>|<payloadByteLength>|<payload>\n
```

字段	类型	说明
msgType	int	消息类型编号（见 § 3.2）
payloadByteLength	int	payload 字段的 UTF-8 字节数（不含 和 \n）
payload	String	实际负载数据，内部可含 分隔符

示例：

```
2|17|a0|a1||12345678|0
```

16.2. B.2 消息帧解析规则

1. 从 TCP 流中读取直到 \n，得到一行文本 以 | 分割，取第 1 段为 msgType 以 | 分割，取第 2 段为 payloadByteLength 剩余部分（第 3 段起，用 | 连接还原）为 payload 校验：payload 的 UTF-8 字节数必须等于 payloadByteLength，否则丢弃并返回 ERROR

解析参考实现（Java）：

```
public static int parseMsgType(String line) {
    return Integer.parseInt(line.split("\\|")[0]);
}

public static String parsePayload(String line) {
    String[] parts = line.split("\\|", 3);
    if (parts.length < 3) return "";
    int declaredLen = Integer.parseInt(parts[1]);
    String payload = parts[2];
    if (payload.getBytes("UTF-8").length != declaredLen) {
        return null; // 帧损坏
    }
    return payload;
}
```

设计理由：采用换行分隔而非纯长度前缀，是因为课程场景下可用 telnet 手动调试，且 payload 内 | 无需转义。payloadByteLength 的首要作用是校验帧完整性，次要作用是允许 payload 包含换行符（当前未使用此特性）。

16.3. B.3 消息类型目录

编号	常量名	方向	说明
1	MSG_LOGIN	C → S	客户端登录 / 加入游戏
2	MSG_MOVE	C ↔ S	走子提交与广播确认
3	MSG_GAME_STATE	S → C	游戏状态变更通知

编号	常量名	方向	说明
4	MSG_ERROR	S → C	错误消息（含错误码）
5	MSG_QUIT	C → S	客户端主动退出
6	MSG_GAME_OVER	S → C	游戏结束通知（含结果与原因码）
7	MSG_BOARD_STATE	S → C	完整棋盘同步（含当前走子方）
8	MSG_DRAW_REQUEST	C ↔ S	提和 / 和棋响应
9	MSG_RESIGN	C ↔ S	认输
10	MSG_CHAT	C ↔ S	文本聊天

实现要求：

- 1 - 7 为必须实现的核心消息
- 8 - 10 为可选扩展，但必须能收到未知消息不崩溃（静默忽略）
- 各组可在本地协议中使用 100+ 的私有消息号，组间通信时不得发送

16.4. B.4 各消息详细格式

16.4.1. B.4.x MSG_LOGIN (1) — 客户端登录 客户端 → 服务器

msgType	len	color	playerName	gameId
1	<len>	<color>	<playerName>	<gameId>

字段	类型	必填	说明
color	int	是	0 = 红方, 1 = 黑方
playerName	String	是	玩家昵称（建议不含 ）
gameId	String	否	指定游戏 ID；空 = 自动匹配

示例：

msgType	len	color	playerName	gameId
1	12	0	张三	(空)
1	20	1	李四	a1b2c3d4

行为约定：

- gameId 为空时，服务器将客户端放入匹配池，凑齐双方后自动开局
- gameId 非空时，服务器查找指定游戏；若不存在则返回 ERROR
- color 为客户端偏好，服务器可覆盖（如先到先得）

16.4.2. B.4.x MSG_MOVE (2) — 走子

客户端 → 服务器（走子请求）

msgType len source destination type turnStartTime isFlipOnly

2 <len> <source><destination><type><turnStartTime><isFlipOnly>

服务器 → 双方（广播确认）

格式同上，服务器在广播前完成：

1. 合法性校验（着法规则 + 路径有效性）
2. 将军检测（走子后己方不能处于被将状态）
3. 若暗子首次翻开，用服务器预生成类型覆盖 type
4. 用服务器当前时间覆盖 turnStartTime

字段	类型	必填	说明
source	String	是	原坐标，如 "b3"
destination	String	是	目标坐标，如 "b4"
type	int 或空	条件	暗子翻开时由服务器填入 0 - 6；否则为空字符串
turnStartTime	long	否	客户端时间戳，服务器忽略并覆盖
isFlipOnly	int	是	1 = 原地翻子，0 = 普通移动

示例：

msgType	len	source	destination	type	turnStartTime	isFlipOnly	说明
2	10	b1	b3		0	0	正常走子 (type 为空， turnStartTime 占位 = 0)
2	10	a0	a0		0	1	原地翻子 a0 位置的暗子
2	20	c4	e4	3	1700000000	0	翻出类型 3 (炮)，时间 戳会被覆盖

16.4.3. B.4.x MSG_GAME_STATE (3) — 游戏状态

仅服务器 → 客户端，通过首个字段区分子类型。

子类型 1: LOGIN_ACK（登录确认）

msgType len subType gameId assignedColor status

3 <len> LOGIN_ACK <gameId><assignedColor><status>

字段	说明
LOGIN_ACK	固定字面量
gameId	分配的游戏 ID
assignedColor	服务器分配的阵营 (0 = 红, 1 = 黑)
status	WAITING 或 PLAYING

服务器收到 LOGIN 后立即回复。

子类型 2: GAME_START (开局广播)

msgType	len	subType	firstMoveColor
3	<len>	GAME_START	<firstMoveColor>

字段	说明
GAME_START	固定字面量
firstMoveColor	0 = 红方先手

双方到齐且棋盘初始化后广播。客户端收到后进入对弈状态。

子类型 3: TURN_CHANGE (回合切换)

msgType	len	subType	currentTurnColor
3	<len>	TURN_CHANGE	<currentTurnColor>

每次合法走子后广播，通知当前轮到谁走。

状态值枚举：

WAITING	等待对手加入
PLAYING	对弈进行中
RED_WIN	红方获胜
BLACK_WIN	黑方获胜
DRAW	和棋
TIMEOUT	超时判负

16.4.4. B.4.x MSG_ERROR (4) — 错误消息

仅服务器 → 客户端

msgType	len	errorCode	errorDescription
4	<len>	<errorCode>	<errorDescription>

字段	说明
errorCode	整数错误码（见下表）

字段	说明
errorDescription	人类可读的中文错误描述

错误码表:

编码	含义
100	未知错误
101	非法着法（棋子移动规则违反）
102	移动路径被阻挡
103	同色吃子（目标为己方棋子）
104	蹩马腿
105	塞象眼
106	走子后己方被将军
107	不是你的回合
108	游戏未在进行中
109	暗子已翻开，不可重复翻子
110	源位置无棋子
111	消息格式错误（帧解析失败）
112	重复登录
200	游戏房间不存在
201	游戏房间已满
202	所选颜色已被占用

16.4.5. B.4.x MSG_QUIT（5）— 退出
客户端 → 服务器

msgType	len
5	<len>

服务器收到后：退出方判负 → 广播 GAME_OVER → 关闭连接。

16.4.6. B.4.x MSG_GAME_OVER（6）— 游戏结束
仅服务器 → 客户端

msgType	len	winner	reasonCode	reasonDescription
6	<len>	<winner>	<reasonCode>	<reasonDescription>

字段	类型	说明
winner	int	0 = 红胜, 1 = 黑胜, -1 = 和棋
reasonCode	int	结束原因码 (见下表)
reasonDescription	String	人类可读的原因描述

示例:

msgType	len	winner	reasonCode	reasonDescription
6	27	0	0	红方将死黑方获胜
6	14	-1	6	40 回合无吃子, 和棋
6	16	1	3	黑方认输, 红方胜

游戏结束原因码:

编码	原因	胜方	说明
0	将死 (Checkmate)	对方	被将军且无任何合法着法可解
1	困毙 (Stalemate)	对方	未被将军但无任何合法着法可走
2	超时 (Timeout)	对方	单步超过 65 秒未走子
3	认输 (Resign)	对方	主动认输
4	断线 (Disconnect)	对方	对手断开 TCP 连接
5	吃将获胜	吃将方	对方未应将, 己方直接吃掉将/帅 (作业明确允许)
6	40 回合无吃子	无 (和棋)	连续 80 个半步无吃子
7	长将/长捉判负	对方	同局面重复 ≥ 6 次, 且非兵卒长捉
8	兵卒长捉和	无 (和棋)	兵卒长捉导致局面重复 ≥ 6 次
9	协议和棋	无 (和棋)	双方同意和棋

16.4.7. B.4.x MSG_BOARD_STATE (7) — 棋盘同步

仅服务器 → 客户端

msgType	len	currentTurn	rows
7	<len><currentTurn>		<row0> <row1> ... <row9>

棋盘行编码:

- 共 10 行, 以 | 分隔 (与 currentTurn 组成 11 段, 以 | 切分 payload)

- row0 = 棋盘最顶行（显示行号 9，黑方底线）
- row9 = 棋盘最底行（显示行号 0，红方底线）
- 每行 9 个 cell，以 , 分隔

Cell 编码规则：

Cell 值	含义
.	空位（该格无棋子）
0 + type	红方已翻开棋子，type 为 0 - 6（例：01 = 红车）
1 + type	黑方已翻开棋子，type 为 0 - 6（例：13 = 黑炮）
0?	红方暗子（未翻开）
1?	黑方暗子（未翻开）

完整示例（开局棋盘，currentTurn=0 红方行棋。帧格式：7|len|0|<row0>|...|<row9>):

	a	b	c	d	e	f	g	h	i
9	1?	1?	1?	1?	10	1?	1?	1?	1?
8	.	.	.	.	.	.	.	.	.
7	.	1?	.	.	.	.	.	1?	.
6	1?	.	1?	.	1?	.	1?	.	1?
5	.	.	.	.	.	.	.	.	.
4	.	.	.	.	.	.	.	.	.
3	0?	.	0?	.	0?	.	0?	.	0?
2	.	0?	.	.	.	.	.	0?	.
1	.	.	.	.	.	.	.	.	.
0	0?	0?	0?	0?	00	0?	0?	0?	0?

Table 3: BOARD_STATE 开局帧（len=211，currentTurn=0）。e9=10（黑将）、e0=00（红帅）
开局即明；0?=红方暗子，1?=黑方暗子，.=空位

16.4.8. B.4.x MSG_DRAW_REQUEST (8) — 提和 双向 (C → S 或 S → C)

客户端提和：

msgType	len	action
8	<len>	OFFER

对方同意：

msgType	len	action
8	<len>	ACCEPT

对方拒绝:

msgType	len	action
8	<len>	DECLINE

服务器收到 ACCEPT 后广播 GAME_OVER (原因码 = 9, 协议和棋)。收到 DECLINE 后仅转发, 棋局继续。

16.4.9. B.4.x MSG_RESIGN (9) — 认输

客户端认输:

msgType	len
9	<len>

服务器通知:

msgType	len	color
9	<len>	<color>

服务器收到认输后立即广播 GAME_OVER (原因码 = 3)。

16.4.10. B.4.x MSG_CHAT (10) — 聊天

双向

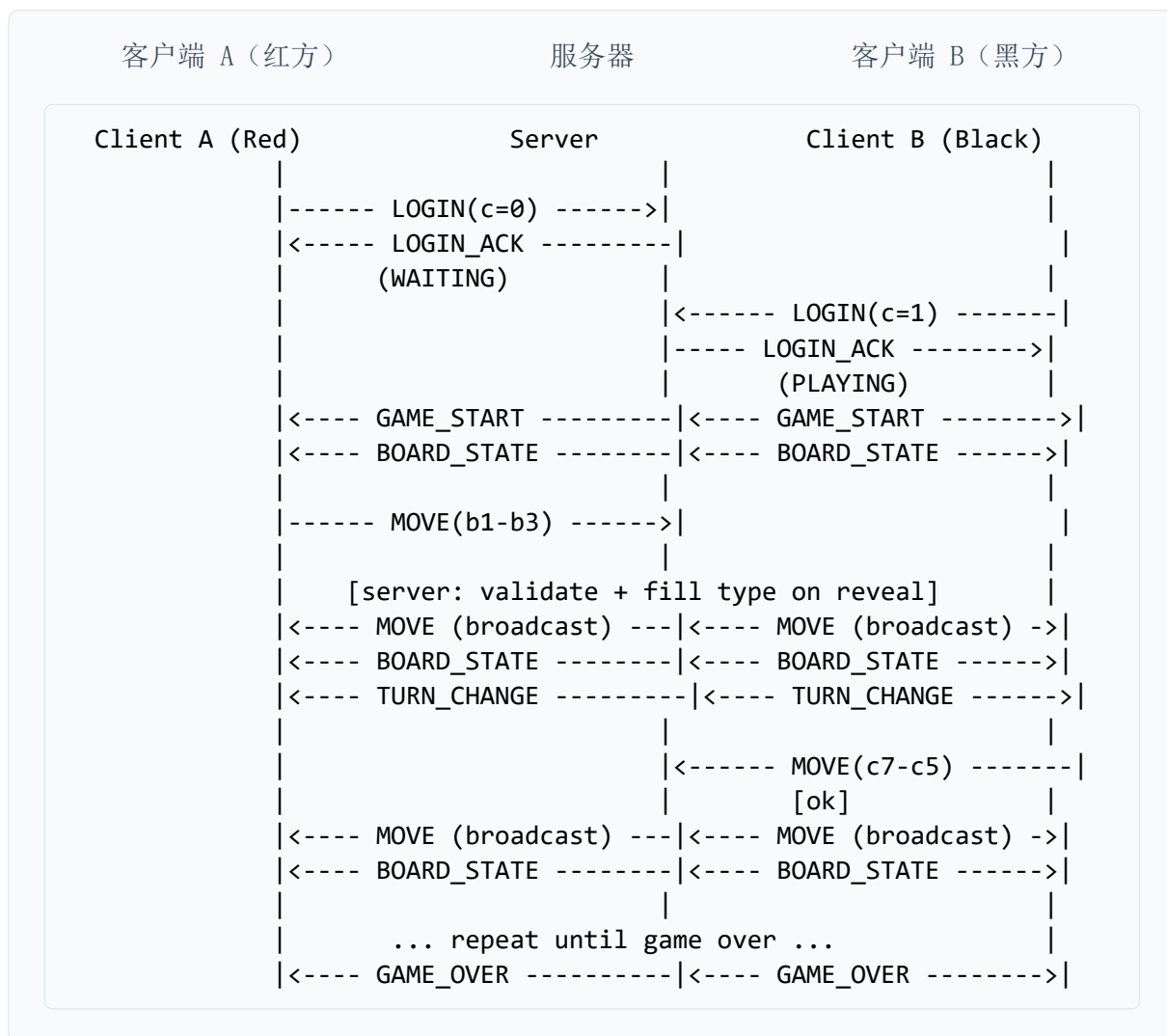
msgType	len	playerColor	playerName	message
10	<len>	<playerColor>	<playerName>	<message>

不影响棋局状态, 服务器仅负责转发。

16.5. B.7 TCP 典型通信时序图

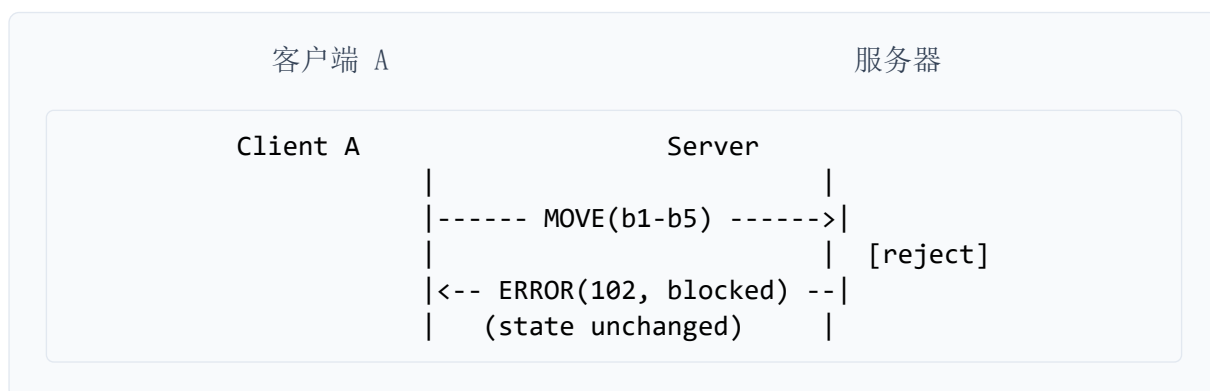
时序图竖线 | 表示各方连接; --> / <-- 表示消息方向。图内为等宽英文标签以保证对齐。

16.5.1. B.7.1 正常对弈时序



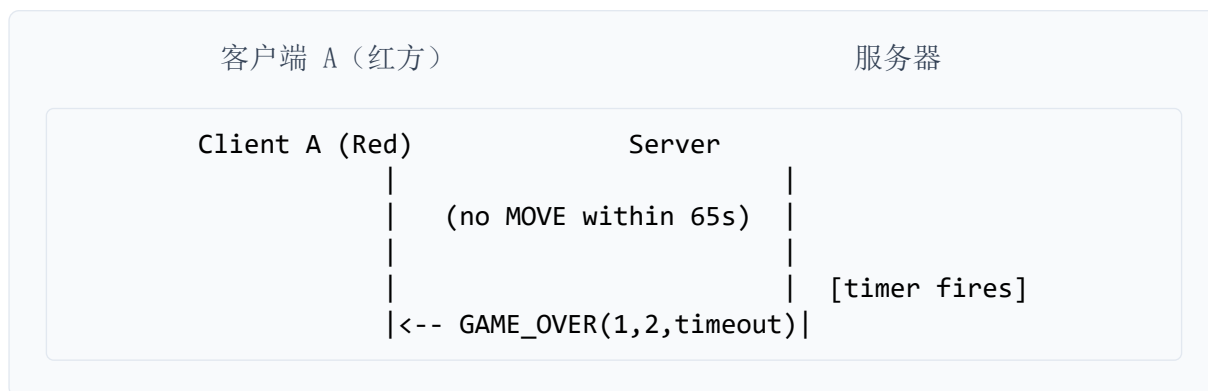
Listing 7: 从登录到终局的消息序列（客户端 A 红方、服务器、客户端 B 黑方）

16.5.2. B.7.2 非法着法被拒绝



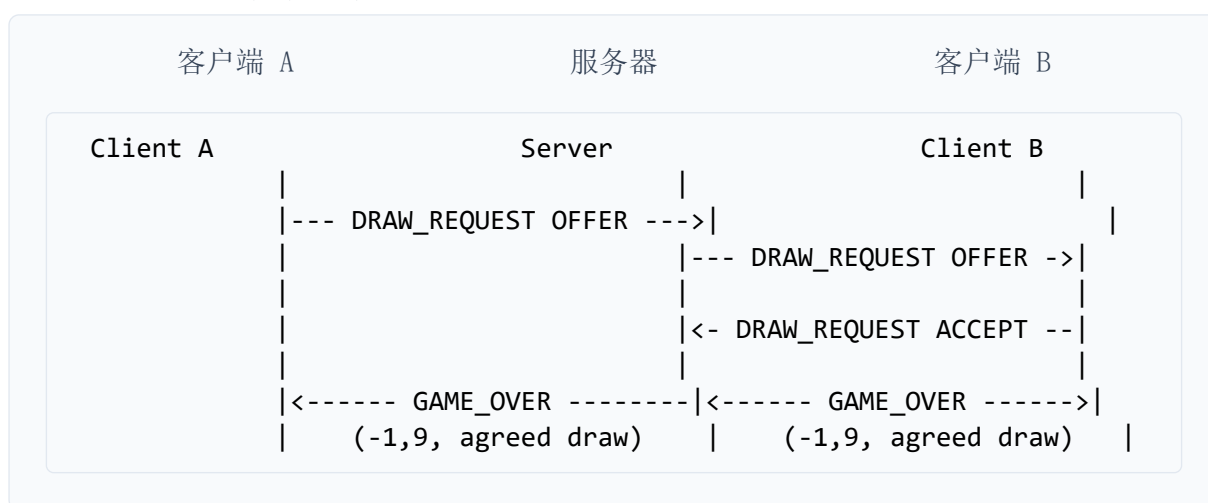
Listing 8: 非法着法被拒绝：棋局状态不变，仍轮到客户端 A

16.5.3. B.7.3 超时判负



Listing 9: 超时判负: 红方 65 秒内未走子, 服务器定时器触发后广播 GAME_OVER

16.5.4. B.7.4 提和流程



Listing 10: 提和流程: 双方 OFFER 后一方 ACCEPT, 服务器广播和棋 GAME_OVER

16.5.5. B.7.5 认输流程



Listing 11: 认输流程: 客户端 A 认输, 服务器通知 B 并广播 GAME_OVER

16.6. B.8 TCP 消息快速参考卡片

类型	方向	payload 格式
LOGIN	C→S	<color>\ <name>\ <gameId>
MOVE	双边	<src>\ <dst>\ <type>\ <time>\ <flip>

类型	方向	payload 格式
GAME_STATE	S→C	LOGIN_ACK\ <id>\ <color>\ <status> / GAME_START\ ... / TURN_CHANGE\ ...
ERROR	S→C	<code>\ <msg>
QUIT	C→S	(空)
GAME_OVER	S→C	<winner>\ <reasonCode>\ <desc>
BOARD_STATE	S→C	<turn>\ <r0>;...;<r9>
DRAW_REQUEST	C↔S	OFFER / ACCEPT / DECLINE
RESIGN	C↔S	(空) 或 <color>
CHAT	双边	<color>\ <name>\ <msg>

17. 附录 C：版本历史

版本号方案：对外正式版本自 v0.0.0（2026-05-15）起算。此前 2026-05-08～05-12 为项目组内部草案，不纳入对外版本序列。

版本	日期	主要变更
		协议版本体系基线（自本日起从 0.0.0 起算）； TCP 文本帧、UTF-8/LF、坐标系（行 9-0、列 a-i）与 Move 字段；
v0.0.0	2026-05-15	7 种核心消息类型与组内 Reference Server 联调通过； BOARD_STATE/Cell 编码（0?/1?/明子）；ERROR 错误码表（100-112）； 暗子服务器随机 type；超时判负与断线处理说明
v1.2	2026-05-18	新增 GAME_STATE 子类型（LOGIN_ACK、GAME_START、TURN_CHANGE）； 起草 MSG_DRAW_REQUEST、MSG_RESIGN、MSG_CHAT 扩展消息； 编写虚拟类型表与明士/明象强化规则说明； 完成与第二组客户端首次互通测试
v1.3	2026-05-20	统一 payload 长度校验规则；补充 GAME_OVER 原因码（0-9）； 增加 40 步无吃子和、长将/长捉判例说明； 整理棋谱记谱格式与组间联调检查清单初版

版本	日期	主要变更
		全面重构为现行权威版本；完整帧格式（msgType len payload）与解析伪代码；
v2.0	2026-05-22	正式纳入 8 - 10 号消息（提和、认输、聊天）； 完善错误码/原因码体系、典型通信时序图、边界判例； 扩展 BOARD_STATE 完整示例与逐行解析； 定稿虚拟类型表、多盘对弈约定及待老师确认问题列表； 本文档作为组间互操作唯一参考
v3.0	2026-05-29	对齐课程《2026 大作业公共接口》：WebSocket + JSON 升为正文主协议（第 6 章）； 原 v2.0 TCP 协议完整迁移至附录 B；默认端口 8887； 新增 WS 联调清单条目
v3.1	2026-05-30	在 v2.0 全文保留基础上扩展（27 条保留自检清单，§ 14 实现状态标注）； 补全 C→S/S→C 字段级说明与老师原文 JSON 示例； 附录 B 含 MSG 1 - 10、BOARD_STATE/Ce11 编码、TCP 五组时序图